基础算法 46 题 · 总复习

> 这份总结不是题解汇编，而是把 46 题**重新压成一套可复用的判断流程**：
> 拿到题先看什么 → 想到哪一章 → 套哪个骨架 → 哪里最容易错。
> 代码骨架都取自本目录 `.cpp带注释版本/` 里你自己写过的写法，方便对照。

---

0. 一张图：拿到题的 60 秒

```
读题
├─ 数据范围是多少？ ──→ 反推复杂度上限（见 0.1 表）
├─ 要回答什么？
│    ├─ 多次「区间求和」          → 前缀和
│    ├─ 多次「区间加、最后统一查」→ 差分
│    ├─ 「最小的最大值 / 最大的最小值」→ 二分答案
│    ├─ 「连续一段满足某性质」    → 双指针 / 滑动窗口
│    ├─ 「有多少种方案 / 路径」   → 递推（有拓扑序）或 DFS（要枚举全部）
│    ├─ 「最少几步」              → BFS（边权相同）
│    └─ 值域 1e9 但只有 n 个数    → 离散化（再配差分或树状数组）
└─ 写之前先定：下标从 0 还是 1、要不要 long long、边界 n=1 怎么办
```

0.1 数据范围 → 允许的复杂度

N 的上限	可以用的复杂度	典型做法
≤ 10	O(N!)	全排列、暴搜
≤ 20	O(2^N)	子集枚举、状压
≤ 300	O(N^3)	三重循环、Floyd
≤ 2000	O(N^2)	双重循环、二维 DP
≤ 1e5 ~ 1e6	O(N log N)	排序、二分、双指针、树状数组
≤ 1e8	O(N)	线性扫描、前缀和
≥ 1e9	O(√N) / O(log N) / O(1)	试除、快速幂、公式

这条表是硬的：`N ≤ 1e5` 时写双重循环 = 1e10 次，必 TLE。46 题里 P1873、P2440、P1638、P3029、P1908 全部卡在这条线上。

---

1. 枚举（01~05）

本质：答案空间小，且"给一个候选就能立刻判断行不行"。

触发信号：`n ≤ 100`、要"选几个数/几组"、没有明显单调性。

```
// P6437 选三个数和不超过 m：三层枚举，内层可以直接剪枝
for (int i = 0; i < n; i++)
    for (int j = i + 1; j < n; j++)
        for (int t = j + 1; t < n; t++)
            if (a[i] + a[j] + a[t] <= m) ans = max(ans, a[i] + a[j] + a[t]);
```

本库五题：P1179 数位统计（`t%10` / `t/10` 拆位）；P2637 拍卖（**降序排序后**枚举"以第 i 高价成交"，收入 `(i+1)*p[i]`，用 `>=` 取到并列时的最低价）；P1085 打擂台；P6437 三层枚举；P8834 二重枚举比乘积。

坑：
- 枚举变量被内层循环改掉（`int t = i` 保护）。
- 只存"满足条件的候选"再求 max，别忘了候选可能为空（P6437 保证有解才敢 `b[0]`）。

---

2. 模拟（06~10）

本质：把文字规则逐条翻译成状态转移，考的是"读题不漏条件"。

触发信号：题面在讲故事、给了明确的操作步骤、没有数学结构。

本库五题：
- P2670 扫雷：八方向计数，答案直接写回格子 `ans + '0'`；二维数组**外圈留一圈 0**，省掉越界判断。
- P1601 高精度：`reverse` 后逐位加，`c` 存进位，最后 `if (c) ans += '1'`。
- P2615 幻方：把四条规则**按顺序**写成 if-else 链，第四条里还要判"右上方是否为空"。
- P9198 轴对称：**不要每次重扫整张图**。维护一个 `diff_count`（不相等的镜像对数），每次修改只更新被改的那一对格子。
- P1098 字符串展开：先判"是否真的该展开"（两侧同类且左 < 右），再按 `p1/p2/p3` 三层处理，否则原样输出 `-`。

```
// P9198 的增量维护骨架（这题最值钱的思想）
long long diff = 0;                          // 当前有多少对镜像格不相等
// 修改前：若本来就不相等，先扣掉
if (j != m_j && !isEqual(img[i][j], img[i][m_j])) diff--;
img[i][j].rgb[t] = (img[i][j].rgb[t] + c) % 256;
// 修改后：若变得不相等，再加回来
if (j != m_j && !isEqual(img[i][j], img[i][m_j])) diff++;
cout << (diff == 0 ? "Yes\n" : "No\n");
```

坑：取模 256 要在加完之后做；中间列（`j == m-j+1`）不影响对称性，要跳过。

---

3. 排序（11~15）

本质：排序把"枚举所有组合"降级成"相邻比较"。

触发信号：题目里有"两两"、"最大/最小"、"按某种顺序输出"、多维属性比较。

本库五题：
- P1177 / P1059：`sort` + `unique`（**必须先排序**，`unique` 只去相邻重复）。
- P5143 攀爬者：按 `z` 排序后累加相邻两点欧氏距离。
- P1104 生日：多关键字 cmp，**"输入靠后的人先输出"用 `id` 逆序实现**（这就是稳定性）。
- B3784 演唱会：边读边累加每首歌总欢乐度，再排序取前 m；`find + erase + insert` 处理"最喜欢的那首"。

```
// P1104 的多关键字 cmp：逐层比较，最后用 id 兜底
bool cmp(const Stu& a, const Stu& b) {
    if (a.y != b.y) return a.y < b.y;      // 年
    if (a.m != b.m) return a.m < b.m;      // 月
    if (a.d != b.d) return a.d < b.d;      // 日
    return a.id > b.id;                    // 同生日：后输入的在前
}
```

坑：`sort` 区间左闭右开（`a, a+n`）；`cmp` 里必须严格弱序，别写 `<=`。

---

4. 贪心（16~20）

本质：每步取当前最优，但**必须能说出"为什么不会更差"**。

触发信号：可以按某个指标排序后一路取；"最少几个"、"最大价值"。

本库五题：
- P2240 部分背包：按**单位价值** `v/m` 降序，能整块拿就整块拿，最后一块切开。
- P6832 子弦：答案是**出现次数最多的单个字符**。理由：任何子串出现 k 次，它的首字符至少出现 k 次——单字符能取到这个上界。
- P9517 drink：一段区间覆盖所有 1 → **第一个 1 到最后一个 1** 的长度；没有 1 输出 0。
- P9045 Oranżada：从左往右扫，遇到没见过的品牌就"提到前面"，代价 = 它前面还剩几个没被固定的瓶子 `i - g - 1`。
- P2676 Bookshelf：从高到低叠，累加到 ≥ B 就停。

```
// P9045: g = 已经固定在左边的瓶子数
for (int i = 1; i <= n; i++)
    if (!used[a[i]]) {
        ans += i - g - 1;       // 中间隔着多少个"多余"的瓶子
        g++; used[a[i]] = true;
        if (g == k) { cout << ans; return 0; }
    }
cout << -1;
```

坑：贪心题先找反例；证不出来就换 DP/二分。

---

5. 前缀和 / 差分（21~24）

本质：把"区间操作"变成"端点操作"，O(n) 预处理换 O(1) 查询/修改。

5.1 一维前缀和（P8218）

```
s[i] = s[i-1] + x;              // 建表
cout << s[r] - s[l-1] << '\n';  // 区间 [l,r] 的和
```

`s` 用 `long long`：`1e5 × 1e4` 已到 1e9 量级。

5.2 一维差分（P2367）

```
d[i] = a[i] - a[i-1];           // 用原数组直接构差分
d[x] += z;  d[y+1] -= z;        // 区间 [x,y] 全体 +z
for (int i = 1; i <= n; i++) {  // 最后一次前缀和还原
    a[i] = a[i-1] + d[i];
    ans = min(ans, a[i]);
}
```

`d[y+1]` 会用到 n+1，数组**开到 n+2**。

5.3 二维前缀和（P2004）

```
sum[x][y] = mp[x][y] + sum[x-1][y] + sum[x][y-1] - sum[x-1][y-1];
// 查询以 (x1,y1) 为左上、(x2,y2) 为右下的矩形
total = sum[x2][y2] - sum[x1-1][y2] - sum[x2][y1-1] + sum[x1-1][y1-1];
```

答案可能全负 → `maxSum` 初始化为 `-1e9`，不能是 0。

5.4 二维差分（P3397）

```
d[x1][y1]++;  d[x1][y2+1]--;
d[x2+1][y1]--; d[x2+1][y2+1]++;              // 四角标记
d[i][j] += d[i-1][j] + d[i][j-1] - d[i-1][j-1]; // 再求二维前缀和还原
```

坑：四角标记的符号是 `+ - - +`，写反一处整张图就错；下标从 1 开始时，`x2+1` 最大到 n+1。

---

6. 二分（25~29）

两条完全不同的路，别混：

6.1 二分查找（在有序表里找位置）

```
// P2249 找「第一次出现」的位置：答案落在左半区
int l = 1, r = n;
while (l < r) {
    int mid = l + (r - l) / 2;
    if (a[mid] >= q) r = mid;       // 还能更左
    else l = mid + 1;
}
if (a[l] == q) cout << l; else cout << -1;
```

- P1678 用 `lower_bound` 找第一个 ≥ 估分的位置，再和**它前一个**比谁更近；注意 `pos == 0` 和 `pos == m` 两种贴边情况。

6.2 二分答案（答案单调，判定 O(n)）

```
// P1873 砍树 / P2440 木材加工 通用骨架
while (L <= R) {
    long long mid = L + (R - L) / 2;
    if (check(mid)) { ans = mid; L = mid + 1; }  // 可行 → 试着更大
    else R = mid - 1;
}
```

`check` 里一旦累加超过需求就 `return true`——**既提速又防溢出**（P1873 的 `sum` 可能到 1e18 量级）。

6.3 实数二分（P1024）

```
for (int i = -100; i < 100; i++) {          // 逐米枚举，保证每段最多一个根
    double l = i, r = i + 1, fl = f(l), fr = f(r);
    if (fl == 0) cout << l << " ";           // 根正好落在整数点
    else if (fl * fr < 0) {                  // 区间内有根 → 二分逼近
        while (r - l > 1e-4) {
            double mid = (l + r) / 2, fm = f(mid);
            if (fl * fm <= 0) { r = mid; fr = fm; }
            else { l = mid; fl = fm; }
        }
        cout << l << " ";
    }
}
```

两个细节：`f(x)` 用**秦九韶**（`((a*x+b)*x+c)*x+d`）少算乘法；末尾单独判 `f(100) == 0`。

坑：`check` 不单调就全错；`mid` 写 `(L+R)/2` 在 1e18 时会溢出，用 `L+(R-L)/2`。

---

7. 双指针（30~34）

本质：用两个指针的**单调移动**替掉一层循环。

7.1 同向（滑动窗口）——"连续一段满足条件"

```
// P1638 逛画展：覆盖全部 m 位画家的最短区间
while (r < n) {
    r++; if (b[a[r]] == 0) cnt++;  b[a[r]]++;
    while (cnt == m) {                       // 条件满足 → 尝试收缩左边界
        if (r - l + 1 < ans) { ans = r-l+1; ansL = l; ansR = r; }
        b[a[l]]--; if (b[a[l]] == 0) cnt--;
        l++;
    }
}
```

- **答案在 `while` 里面更新**才是"最短"；在 outside 更新就变成"最后一个满足的"。
- P3029 同理，只是窗口代价换成坐标差 `a[r].pos - a[l].pos`。

7.2 相向 / 快慢——"和太小就加、太大就减"

```
// P1147 连续正整数和
int l = 1, r = 1, sum = 0;
while (l <= m / 2) {
    if (sum < m)      { sum += r; r++; }
    else if (sum > m) { sum -= l; l++; }
    else { cout << l << " " << r-1 << endl; sum -= l; l++; }
}
```

7.3 前后缀 + 双指针（P3143）

两个柜子 = 找一个**分割点** `i`，答案是 `pre[i] + suf[i+1]`：

```
pre[i] = max(pre[i-1], i - j + 1);   // j 向右滑，保持 a[i]-a[j] <= k
suf[i] = max(suf[i+1], j - i + 1);   // j 向左滑，保持 a[j]-a[i] <= k
```

这是"**单区间最优 → 双区间最优**"的标准套路：先对每个前缀/后缀求最优，再拼。

---

8. 递推（35~38）

本质：有拓扑序 + 无后效性，用已知的小规模推大规模。

题	状态	转移
P1255 数楼梯	`f[i]` = 走到第 i 阶的方案数	`f[i] = f[i-1] + f[i-2]`（斐波那契）
P1176 路径计数	`dp[i][j]` = 到 (i,j) 的方案数	`dp[i][j] = dp[i-1][j] + dp[i][j-1]`，障碍置 0
P1028 数的计算	`f[i]` = 以 i 开头的合法数列数	`f[i] = f[i-1] + (i 偶 ? f[i/2] : 0)`
P1216 数字三角形	`a[i][j]` = 从该点到底的最大和	`a[i][j] += max(a[i+1][j], a[i+1][j+1])`

```
// P1216：自底向上，越往上路径越少，最后 a[0][0] 就是答案
for (int i = r - 2; i >= 0; i--)
    for (int j = 0; j <= i; j++)
        a[i][j] += max(a[i+1][j], a[i+1][j+1]);
```

P1255 要特别注意：N ≤ 5000，斐波那契早就爆 `long long`，必须**高精度 + 滚动数组**（只留 3 个 vector，`f[i%3]`）。

```
vector<int> f[3] = { {1}, {1}, {} };
for (int i = 2; i <= n; i++)
    f[i % 3] = add(f[(i-1) % 3], f[(i-2) % 3]);
```

坑：初值 `f[1] = 1`（P1028）；取模要在**每一步**都做（P1176 的 `% 100003`）。

---

9. 搜索（39~44）

分岔口：要**枚举所有方案 / 数方案 / 数连通块** → DFS；要**最短步数** → BFS。

9.1 DFS 回溯三件套

```
void dfs(int step) {
    if (step == n + 1) { 输出; return; }      // 1. 终止条件
    for (int i = 1; i <= n; i++)
        if (!used[i]) {
            used[i] = true;   ans[step] = i;  // 2. 做选择
            dfs(step + 1);
            used[i] = false;                  // 3. 撤销选择（恢复现场）
        }
}
```

9.2 网格 DFS（P1605 迷宫 / P1596 连通块 / P1162 填涂 / P1644 跳马）

```
int dx[] = { -1, 1, 0, 0 }, dy[] = { 0, 0, 1, -1 };   // 四方向
// 八方向就用两重 -1..1 的循环，或写 8 组 dx/dy
```

- **数方案**（P1605、P1644）：到终点 `ans++` 后 `return`，路径上的标记要**恢复**。
- **数连通块**（P1596）：进去就把格子改成 `.`（"染色"），**不恢复**，主函数里每遇到一个 `W` 就 `ans++` 再灌水。
- **P1162 填涂颜色**：反过来想——先从外圈 (0,0) 灌水标记"圈外的 0"，剩下没被标记的 0 就是圈内，输出 2。**数组四周留一圈 0** 是关键。

9.3 BFS（P1746 离开中山路）

```
queue<PII> q;  q.push({x1, y1});  d[x1][y1] = 0;   // d 兼作 vis
while (!q.empty()) {
    auto t = q.front(); q.pop();
    int x = t.first, y = t.second;
    if (x == x2 && y == y2) return d[x][y];        // 弹出时判终点
    for (int i = 0; i < 4; i++) {
        int nx = x + dx[i], ny = y + dy[i];
        if (合法 && d[nx][ny] == -1 && m[nx][ny] == '0') {
            d[nx][ny] = d[x][y] + 1;               // 入队即标记（别等弹出）
            q.push({nx, ny});
        }
    }
}
```

`memset(d, -1, sizeof d)` 一步完成"未访问"初始化。

坑：DFS 忘了恢复现场 → 方案数偏小；BFS 在弹出时才标记 → 同一格重复入队、内存爆。

---

10. 离散化（45~46）

本质：值域 1e9 装不进数组，但个数只有 n（≤2e4 / 5e5）→ 映射成 1..m 的下标。

```
sort(vals.begin(), vals.end());
vals.erase(unique(vals.begin(), vals.end()), vals.end());   // 三步：排序→去重
int r = lower_bound(vals.begin(), vals.end(), x) - vals.begin() + 1;  // → 映射
```

10.1 离散化 + 差分（P1496 火烧赤壁）

```
// 收集所有端点 → 排序去重 → 每个区间在 u 处 +1、v 处 -1
diff[u]++;  diff[v]--;
for (int i = 0; i < m - 1; i++) {
    cur += diff[i];
    if (cur > 0) sum += vals[i+1] - vals[i];   // 有火的"基本区间"长度
}
```

注意题目是**左闭右开**，所以用 `vals[i+1] - vals[i]` 累加；`long long` 必开（坐标到 2^31）。

10.2 离散化 + 树状数组（P1908 逆序对）

```
int lowbit(int x) { return x & -x; }
void add(int i, int v) { while (i <= n) { bit[i] += v; i += lowbit(i); } }
int sum(int i) { int s = 0; while (i > 0) { s += bit[i]; i -= lowbit(i); } return s; }

for (int i = a.size() - 1; i >= 0; i--) {      // 倒序扫描
    int rank = lower_bound(b.begin(), b.end(), a[i]) - b.begin() + 1;
    ans += sum(rank - 1);                      // 右侧已出现、且比我小的个数
    add(rank, 1);
}
```

- **倒序**的好处：`sum` 里统计到的都是"右边"的数，天然满足 `i < j`。
- `bit` 大小是**去重后的 m**，不是原 n。
- `ans` 用 `long long`：逆序对最多 `n(n-1)/2 ≈ 1.25e11`。

---

11. 三条贯穿全章的主线

主线一：「区间」家族的五个变体

需求	工具	复杂度
多次查区间和（可减）	前缀和	O(1)/次
多次区间加、最后统一查	差分	O(1)/次
连续一段满足某条件	双指针	O(n)
最小/最大可行值	二分答案	O(n log V)
值域过大	离散化	O(n log n)

它们解决的是同一件事的不同变体，**换的是"操作"和"查询"的顺序**。

主线二：排序是万能钥匙

排序之后才可能出现：相邻比较（P2637、P5143、P2676、P3143）、二分（P2249、P1678、P1873、P2440）、双指针（P3029、P3143）、去重（P1059、P1908 的 unique）。**卡住时先问一句：排个序会不会变简单？**

主线三：增量维护，别全量重算

P9198 的 `diff_count`、P2367 的差分、P1638 的 `cnt`、P3029 的 `map` 计数——都是"局部改动、代价 O(1)"。凡是看到"每次操作后询问整体状态"，先想能不能**只更新被改动的部分**。

---

12. 复盘：46 题里最容易错的 8 个点

1. `int` **溢出**：前缀和（P8218）、砍树求和（P1873）、逆序对答案（P1908）、坐标差（P1496）、背包价值（P2240）。看到"求和/乘积/坐标 1e9"直接 `long long`。
2. `d[y+1]` **越界**：差分的右端点要写 `+1`，数组开 `n+2`（P2367、P3397）。
3. `unique` **前没排序**：它只去"相邻"重复（P1059、P1908）。
4. **二分边界写错**：找左界用 `l<r` + `r=mid`；二分答案用 `l<=r` + `ans` 记录（P2249 vs P1873）。
5. **滑动窗口答案更新位置**：在 `while (满足)` 里面更新才是最短（P1638、P3029）。
6. **DFS 忘恢复现场 / BFS 忘标记**：方案数变少 / 重复入队（P1605、P1746）。
7. **下标 0 基 1 基混用**：`cin` 进来是 1..n 就全程用 1 基，循环写 `i<=n`；P2004 因为把行列转置存了，输出时还得反着来 `maxY+1 maxX+1`，非常容易错。
8. **边界没测**：`n=1`（P1496、P1908）、全是 0（P9517 输出 0）、切不出 1cm（P2440 输出 0）、答案全负（P2004 初值 `-1e9`）。

顺手记下的两处代码可改进点

- **P1085 不高兴的津津**：`ans` 初始为 0，且 `max` 取了第 1 天。如果**只有第 1 天**超过 8 小时，后面没有更优的，`ans` 会停在 0，输出 `0`（应为 `1`）。样例恰好第 3 天最大，所以没暴露。把 `ans` 初始成 `1`、或让 `max` 从 `-1` 开始即可。
- **P1746 的 `bfs()`**：函数末尾没有 `return`。题目保证有解时不会走到，但严格说是未定义行为，补一个 `return -1;` 更稳。

---

13. 46 题速查表

#	题	章	一句话解法	复杂度
01	P1179 数字统计	枚举	逐个数拆位统计 2	O(n·位数)
02	P2637 拍卖	枚举	降序排序，枚举成交价 × 人数	O(m log m)
03	P1085 津津	枚举	打擂台找最大 (a+b)	O(1)
04	P6437 JACK	枚举	三层枚举取 ≤m 的最大和	$O(n^3)$
05	P8834 序列	枚举	二重枚举比乘积	$O(n^2)$
06	P2670 扫雷	模拟	八方向数雷，外圈留 0	O(nm)
07	P1601 高精度	模拟	反转后逐位加、进位	O(len)
08	P2615 幻方	模拟	四条规则写成 if-else 链	$O(N^2)$
09	P9198 轴对称	模拟	维护不相等镜像对数	O(nm+q)
10	P1098 展开	模拟	判同类→按 p1/p2/p3 填充	O(len·p2)
11	P1177 排序	排序	`sort`	O(n log n)

#	题	章	一句话解法	复杂度
12	P1059 随机数	排序	`sort + unique`	O(n log n)
13	P5143 攀爬者	排序	按 z 排序累加距离	O(n log n)
14	P1104 生日	排序	多关键字 cmp + id 逆序	O(n log n)
15	B3784 演唱会	排序	累加总和→排序→调最爱	O(an + n log n)
16	P2240 部分背包	贪心	单位价值降序，最后一块切分	O(n log n)
17	P6832 子弦	贪心	答案 = 最多出现的单字符次数	O(n)
18	P9517 drink	贪心	首个 1 到末个 1 的长度	O(n)
19	P9045 Oranżada	贪心	新品牌前移，代价 i-g-1	O(n)
20	P2676 书架	贪心	从高到低叠到 ≥B	O(n log n)
21	P8218 区间和	前缀和	`s[r]-s[l-1]`	O(n+m)
22	P2367 语文成绩	差分	区间加→前缀和还原取 min	O(n+p)
23	P2004 领地选择	二维前缀和	C×C 滑窗	O(NM)
24	P3397 地毯	二维差分	四角 `+ - - +` 再还原	O(n²+m)
25	P2249 查找	二分查找	手写左界二分	O(m log n)
26	P1678 高考志愿	二分查找	`lower_bound` 后比左右邻居	O((m+n)log m)
27	P1024 三次方程	实数二分	逐米判号 + 区间二分	O(100·log)
28	P1873 砍树	二分答案	`check(H)` 单调	O(n log H)
29	P2440 木材	二分答案	`check(len)` 数段数	O(n log L)
30	P1147 连续和	双指针	小了加 r、大了减 l	O(m)
31	P7714 排列排序	双指针	`max==i` 处切块，块长>1 计费	O(n)
32	P3143 钻石	双指针+前后缀	分割点拼 `pre[i]+suf[i+1]`	O(n log n)
33	P1638 逛画展	滑动窗口	覆盖 m 位画家的最短区间	O(n)
34	P3029 奶牛	滑动窗口	覆盖全部品种的最小坐标差	O(n log n)
35	P1255 数楼梯	递推	斐波那契 + 高精度滚动	O(n²)
36	P1176 路径计数	递推	上+左，障碍置 0，取模	O(n²)
37	P1028 数的计算	递推	`f[i]=f[i-1]+(偶? f[i/2]:0)`	O(n)
38	P1216 数字三角形	递推	自底向上取 max	O(r²)
39	P1706 全排列	DFS	`used` 数组回溯	O(n·n!)
40	P1605 迷宫	DFS	四方向计数 + 恢复现场	指数
41	P1596 水坑	DFS	八方向染色计数	O(nm)

#	题	章	一句话解法	复杂度
42	P1746 中山路	BFS	`d` 兼 vis，入队即标记	$O(n^2)$
43	P1162 填涂	DFS 灌水	从外圈灌水，剩下的 0 填 2	$O(n^2)$
44	P1644 跳马	DFS	四个固定方向累计方案	指数
45	P1496 火烧赤壁	离散化	端点差分 + 基本区间长度	O(n log n)
46	P1908 逆序对	离散化+树状数组	倒序 `sum(rank-1)` 再 `add`	O(n log n)

---

14. 下一步学什么

46 题停在"离散化 + 树状数组"这个门槛上，往下的路很清楚：

1. **树状数组进阶**：区间改 + 单点查（差分树状数组）、区间改 + 区间查（两个树状数组）、二维树状数组。
2. **线段树**：区间最值/区间加、懒标记、动态开点。逆序对用线段树也能做，和 P1908 对拍很合适。
3. **DP 系统化**：01 背包 / 完全背包 / 区间 DP / 状压 DP / LIS 的 O(n log n) 优化（P3143、09I 都用到了 LIS 的思想）。
4. **图论**：存图（邻接表）、DFS/BFS 遍历、最短路（Dijkstra / Floyd）、拓扑排序、并查集。
5. **字符串**：哈希、KMP、Trie（P1908 的标签里就出现了 Trie，可以顺手看看）。

复习节奏建议：按章回做，每章挑 1 题**关掉笔记白板重写**；隔 1 周、1 个月各来一遍。能一次写对的，才算真的会了。